Arena Quick Insight

SYNTHETIC / EXAMPLE DATA

Run: 20261106T221003-b19503a8c2
Family: games | Seed: 7

Observed metric	Value
conversion_return	0.1643588
exploitability_gap	0.013997201
knapsack_reward	19
oracle_reward	19

Opponent distributions and utility rules are assumptions; rerun sensitivity scenarios.
Inspect the equilibrium solver's best-response gap before using its mix.

What this means

The table contains observed values from this saved run, with no invented performance estimates. Values from different metrics have different units and must not be ranked together. Synthetic comparisons demonstrate implementation and sensitivity; they do not demonstrate trading edge.

Limitations and next test

Official runtime, API, scoring and dependencies must still be checked. Use the official evaluator and legal data, compare paired seeds, and inspect worst cases. A mock fill model can materially change the ranking of strategies.

Provenance

Source: experiments/20261106T221003-b19503a8c2/result.json
Timestamp: 2026-11-06T22:10:03.237736+00:00
Dataset SHA-256: 1d04f63102e990cc8b7dfe6b82923547bfa25243f46a47db861839cc6a827b07
Source SHA-256: f9a8d24a2cc9e6683898ae67a9df49c04c31f045c8bfe0d9cfa0d35839cf803a
Git: 646637196f157edf831aea394233076164801419; dirty=True

Knapsack

SYNTHETIC / EXAMPLE PUZZLE | knapsack

Run: 20261106T221003-b19503a8c2 | Seed: 7

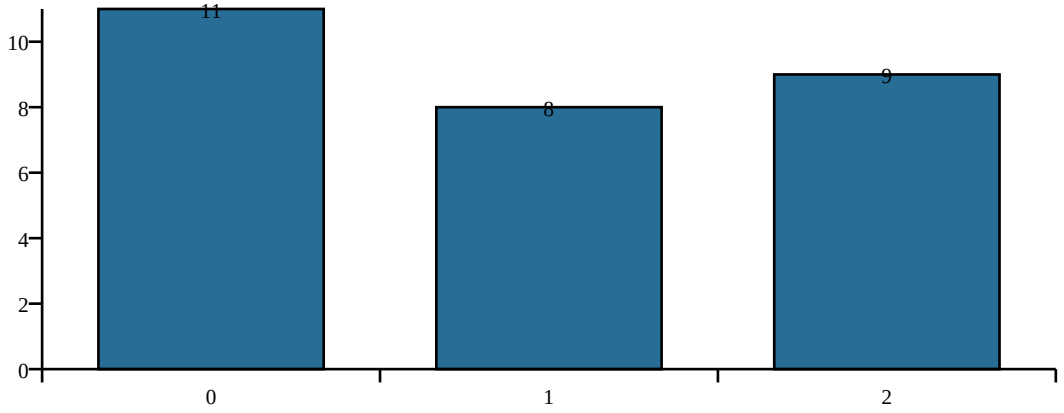
Choose each item at most once; maximize reward within integer budget.

Budget: 10; chosen item indices: [0, 1]

Item	Cost	Reward	Selected?
0	6	11	yes
1	4	8	yes
2	5	9	no

Reward: 19; oracle reward: 19; unused budget: 0

Item rewards (first 12 at most; reward units)



Why: DP compares taking or skipping each item. Enumeration independently checks the tiny instance. Costs and rewards must be additive; indivisible items may be chosen once. Table shows at most eight items.

Source: experiments/20261106T221003-b19503a8c2/result.json | result.puzzles[0]

Next: verify the actual objective and legal actions, then rerun with official inputs and alternative opponent assumptions.

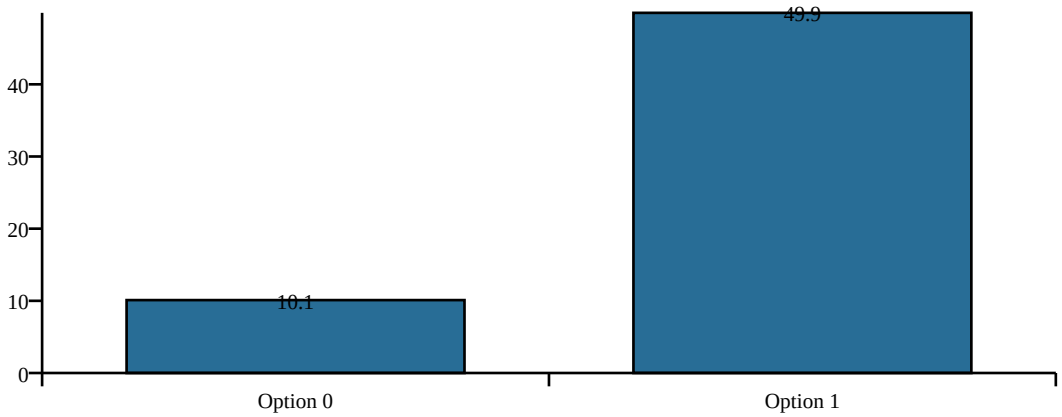
Shared Reward

SYNTHETIC / EXAMPLE PUZZLE | shared_reward

Run: 20261106T221003-b19503a8c2 | Seed: 7

10 independent rivals choose A/B with probabilities 0.9/0.1; rewards 100/80 split equally.

Expected reward per choice (reward units)



Best option index: 1

Option index	Expected payoff
0	10.10101
1	49.904684

Why: a less crowded option can pay more despite a smaller prize. Compute the expectation of reward divided by crowd size, not reward divided by expected crowd size. Independence and rival choice probabilities are assumptions.

Source: experiments/20261106T221003-b19503a8c2/result.json | result.puzzles[1]

Next: verify the actual objective and legal actions, then rerun with official inputs and alternative opponent assumptions.

Sealed Bid Auction

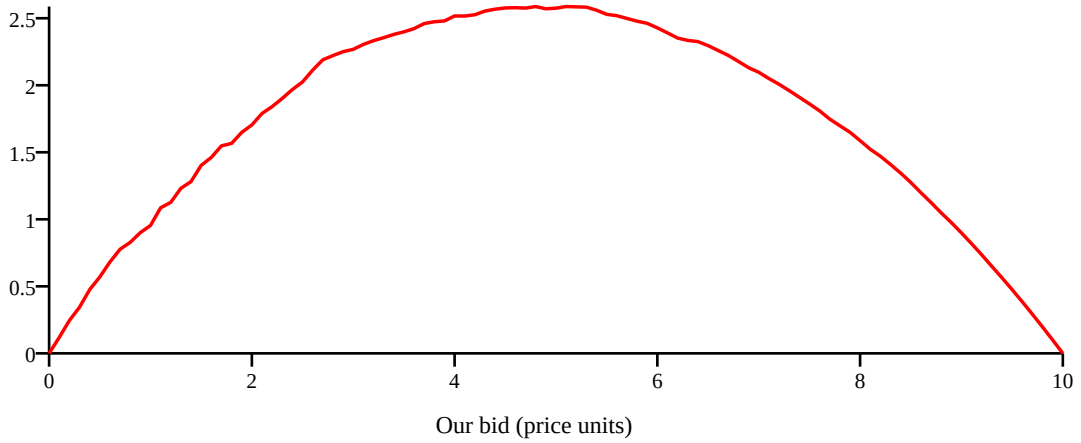
SYNTHETIC / EXAMPLE PUZZLE | sealed_bid_auction

Run: 20261106T221003-b19503a8c2 | Seed: 7

First-price auction, value 10, highest rival bid uniform on [0,10]; empirical grid search.

Payment rule: first_price; tie win probability: 0.5

Empirical expected utility (reward units)



Selected bid	Expected utility under sampled rivals
5.1	2.5872

Why: compare utility over the whole bid grid. A higher bid wins more often but can leave less profit. Monte Carlo sampling and the assumed rival distribution can move the selected bid. Confirm reserve prices, tie rules and payment rules from the real statement.

Source: experiments/20261106T221003-b19503a8c2/result.json | result.puzzles[2]

Next: verify the actual objective and legal actions, then rerun with official inputs and alternative opponent assumptions.

Matrix Game

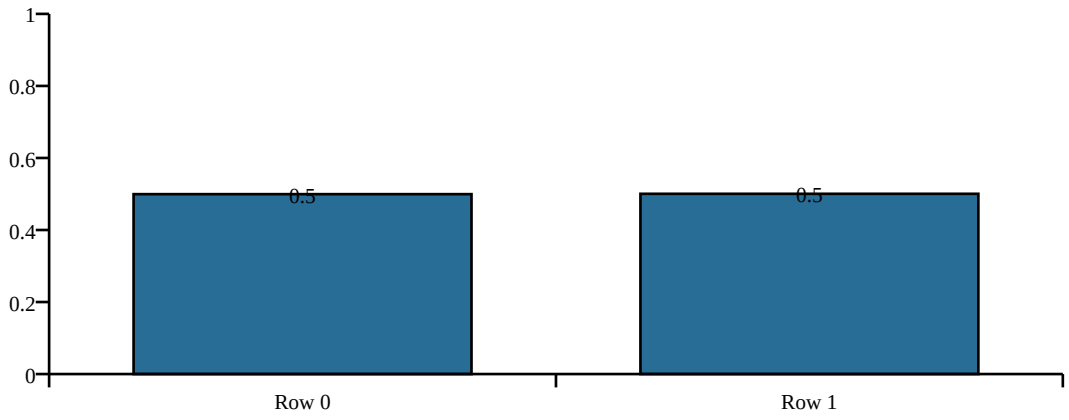
SYNTHETIC / EXAMPLE PUZZLE | matrix_game

Run: 20261106T221003-b19503a8c2 | Seed: 7

Row maximizes, column minimizes the supplied zero-sum payoff matrix.

Payoff matrix	Column payoffs (first six columns)
Row 0	1, -1
Row 1	-1, 1

Row-player mixed strategy (probabilities; first 10)



Method: fictitious_play_approximation

Value: -1.2994802e-05; lower bound: -0.00099980004; upper bound: 0.012997401; best-response gap: 0.013997201

Why: mixing makes the row player harder to exploit. The lower bound is what its mixture guarantees; the upper bound comes from the column mixture. A small gap supports an approximate equilibrium. This applies to two-player zero-sum games. Matrix table shows at most six rows/columns.

Source: experiments/20261106T221003-b19503a8c2/result.json | result.puzzles[3]

Next: verify the actual objective and legal actions, then rerun with official inputs and alternative opponent assumptions.

Conversion Graph

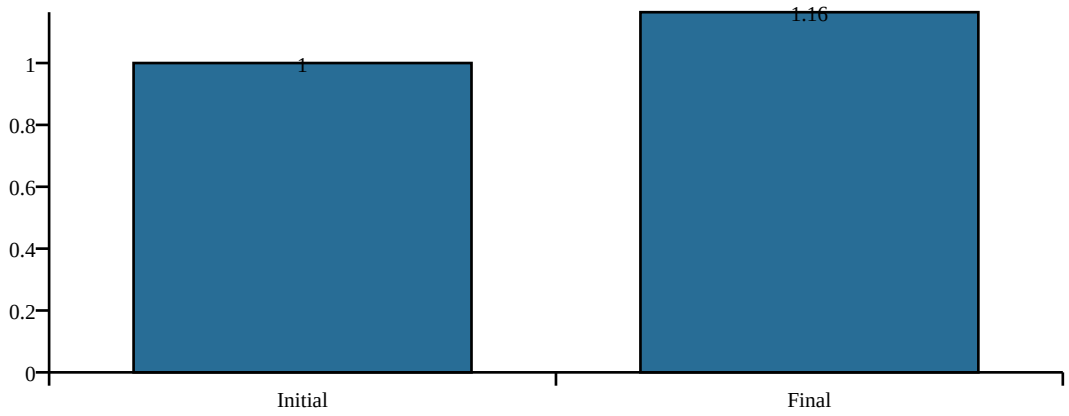
SYNTHETIC / EXAMPLE PUZZLE | conversion_graph

Run: 20261106T221003-b19503a8c2 | Seed: 7

At most 3 conversions A->B->C->A, one percent fee each; maximize final A.

Best path: A -> B -> C -> A

Holdings in starting currency (units)



Initial amount	Final amount	Fee per conversion	Maximum steps
1	1.1643588	0.01	3

Why: bounded dynamic programming compares all reachable currencies after each conversion. Keeping the starting holding allows doing nothing if every cycle loses money. Depth, changing prices and slippage are absent from this toy model.

Source: experiments/20261106T221003-b19503a8c2/result.json | result.puzzles[4]

Next: verify the actual objective and legal actions, then rerun with official inputs and alternative opponent assumptions.

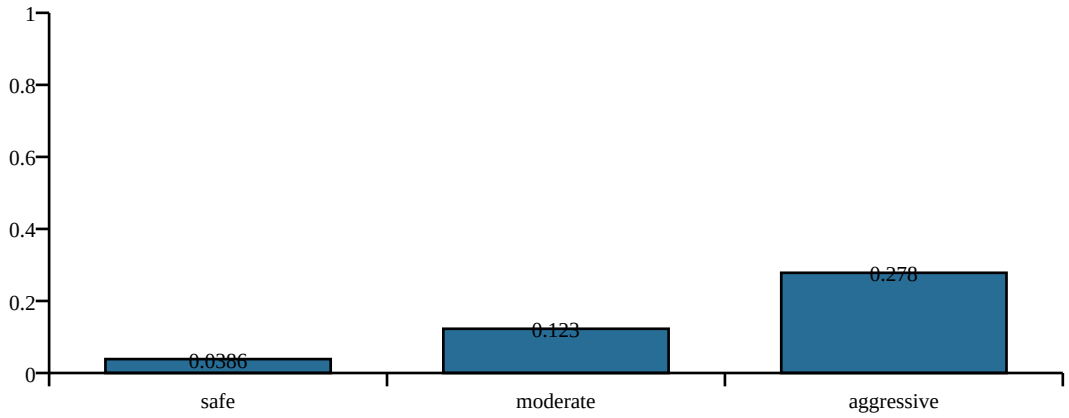
Tournament Risk

SYNTHETIC / EXAMPLE PUZZLE | tournament_risk

Run: 20261106T221003-b19503a8c2 | Seed: 7

Choose a risk level for one final round while trailing by 10; assumed normal returns.

Probability of first place under assumed returns



Risk choice	Expected final score	Probability first	Sampling SE
safe	96	0.0386	0.0027243362
moderate	95	0.1228	0.0046415549
aggressive	93	0.2782	0.0063372669

Best expected score: safe; best probability of first: aggressive; trials: 5000

Why: maximizing average score can choose a different risk level from maximizing the chance of winning. Common random draws compare policies fairly. Sampling SE measures Monte Carlo uncertainty under the assumed distributions; it does not validate those distributions.

Source: experiments/20261106T221003-b19503a8c2/result.json | result.puzzles[5]

Next: verify the actual objective and legal actions, then rerun with official inputs and alternative opponent assumptions.